操作系统

一、程序、进程、线程的关系

> **程序 ≠ 进程**

- **程序**：是**静态的指令集合**（躺在磁盘上的 `.exe` 文件），不占用 CPU 和内存，只是一个"菜谱"。
- **进程**：是**程序的一次执行实例**，是"活的"、运行中的实体。

```
程序（Program） → 可以启动多个进程
                → 每个进程内包含 N 个线程
```

示例：双击 Chrome 3 次 → 产生 3 个进程，它们跑的是同一个 Chrome 程序，但各自独立运行。

关系图（程序 → 进程 → 线程）

```
flowchart LR
    P[程序 Program
──────────
静态指令集合
躺在磁盘上的 .exe 文件
不占CPU/内存] -->|启动多个| Proc1["
进程 1（程序执行实例）
──────────────────
```

本质	资源分配和管理的基本单位
资源	独立地址空间、文件描述符表等
切换开销	非常大（切换页表 / 刷新 TLB / 内核态切换）
独立性	互不影响，一个崩溃不影响其他

```
"]
    P -->|启动| Proc2[进程 2（另一个实例）
──────────────────
资源同样独立
与进程1互不影响]
    Proc1 -->|包含| T1["
线程 1
──────────────────
```

本质	CPU 调度的基本单位
资源	共享代码段、堆、全局变量等
切换开销	非常小（仅换栈 + 寄存器，不切地址空间）
私有	栈、寄存器、TLS
风险	一个线程写坏内存，进程可能崩溃

```
"]
    Proc1 -->|包含| T2[线程 2]
    Proc1 -->|包含| T3[线程 N]

    style P fill:#fff3cd,stroke:#ffc107,stroke-width:2px,color:#000
    style Proc1 fill:#ffe6e6,stroke:#ff6666,stroke-width:2px,color:#000
    style Proc2 fill:#ffe6e6,stroke:#ff6666,stroke-width:2px,color:#000
    style T1 fill:#e6ffe6,stroke:#66cc66,stroke-width:2px,color:#000
    style T2 fill:#e6ffe6,stroke:#66cc66,stroke-width:2px,color:#000
    style T3 fill:#e6ffe6,stroke:#66cc66,stroke-width:2px,color:#000
```

- **程序**：静态的指令集合，不占 CPU 和内存，只是“菜谱”。
- **进程**：程序的一次执行实例，是“资源分配和管理的基本单位”。
- **线程**：CPU 调度的基本单位，是进程中的执行流。

维度	进程（Process）	线程（Thread）
本质	资源分配和管理的基本单位	CPU 调度的基本单位
资源	独立地址空间、文件描述符等	共享代码段、堆、全局变量等
切换开销	大（页表 / TLB / 内核态）	小（仅换栈 + 寄存器）
私有/独立	资源相对独立，互不影响	栈、寄存器、TLS 私有
风险	一个进程崩溃不影响其他进程	一个线程出错可能拖垮整个进程

> ⚠ **重要补充**：线程也有自己**私有**的东西 — **栈（stack）、寄存器（registers）、线程本地存储（TLS）**，不能理解为线程之间所有东西都是共享的！

二、线程间：共享 vs 私有

```
graph TB
    subgraph 共享 ["🔴 所有线程共享（进程级资源）"]
```

```
        direction LR
        A["📄 代码段 .text"]
        B["🌍 全局变量 .data / static"]
        C["📁 堆内存 heap（malloc / new）"]
        D["📂 打开的文件描述符 / socket"]
        E["📂 当前工作目录"]
    end

    subgraph 线程1 ["🟢 线程 1 私有"]
        direction TB
        T1_Stack["📦 栈 stack — 局部变量、函数调用帧"]
        T1_Reg["🔢 寄存器 — 通用寄存器、PC"]
        T1_TLS["🔐 线程本地存储 TLS"]
    end

    subgraph 线程2 ["🔵 线程 2 私有"]
        direction TB
        T2_Stack["📦 栈 stack"]
        T2_Reg["🔢 寄存器"]
        T2_TLS["🔐 线程本地存储 TLS"]
    end

    共享 --- 线程1
    共享 --- 线程2

    style 共享 fill:#ffe6e6,stroke:#ff6666,stroke-width:2px,color:#000
    style 线程1 fill:#e6ffe6,stroke:#66cc66,stroke-width:2px,color:#000
    style 线程2 fill:#e6f0ff,stroke:#6699ff,stroke-width:2px,color:#000
```

五、一句话记忆口诀

> 🏠 **进程是"资源房子"，线程是"干活的人"。**

类比	房子（进程）	人（线程）
关系	各自独立，拆一间不影响隔壁	同住一间房，共享家具（堆、全局变量）
私有物	—	各带随身物品（栈、寄存器、TLS）
切换成本	搬家（切换进程）很贵	换个人干活（切换线程）很便宜
风险	—	一个人乱砸东西，整间房都遭殃